IF3120 - Pembelajaran Mesin

Laporan Tugas Besar 1 - Feedforward Neural Network

Kelompok 37 - MajangLu

Wardatul Khoiroh	13523001
Indah Novita Tangdililing	13523047
Sakti Bimasena	13523053

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

Daftar Isi **2**
A. Deskripsi Persoalan **3**
B. Pembahasan **4**
1. Penjelasan Implementasi 4
a. Deskripsi Kelas 4
b. Forward Propagation 9
c. Backward Propagation 10
2. Hasil Pengujian 12
a. Eksperimen Hyperparameter Width 12
b. Eksperimen Hyperparameter Depth 13
c. Eksperimen Hyperparameter Fungsi Aktivasi 14
d. Eksperimen Hyperparameter Learning Rate 20
e. Eksperimen Pengaruh Regularisasi 23
f. Eksperimen Pengaruh Inisialisasi Bobot 26
g. Eksperimen Uji Perbandingan 29
C. Kesimpulan dan Saran **30**
D. Lampiran **32**

A. Deskripsi Persoalan

Tugas Besar 1 mata kuliah IF3270 Pembelajaran Mesin semester II tahun ajaran 2025/2026 berfokus pada pemahaman mendalam mengenai arsitektur Feedforward Neural Network (FFNN) melalui implementasi mandiri. Persoalan utama dalam tugas ini adalah membangun sebuah modul jaringan saraf tiruan yang fleksibel tanpa menggunakan library machine learning untuk logika intinya. Modul harus mampu menerima konfigurasi jumlah layer dan neuron yang variatif, serta mengintegrasikan berbagai fungsi aktivasi seperti Linear, ReLU, Sigmoid, Tanh, dan Softmax untuk mendukung proses pembelajaran data.

Selain struktur dasar, implementasi ini mencakup mekanisme pembelajaran melalui algoritma feed-forward dan back-propagation. Tantangan teknis dalam persoalan ini terletak pada penurunan matematis gradien untuk setiap fungsi aktivasi dan fungsi loss (seperti MSE, Binary Cross-Entropy, dan Categorical Cross-Entropy) guna memperbarui bobot jaringan secara akurat. Pengguna juga diwajibkan menyertakan metode regularisasi L1 dan L2 sebagai teknik untuk mengontrol kompleksitas model dan mencegah terjadinya overfitting[1] selama proses pelatihan.

Setelah modul berhasil diimplementasikan, persoalan berlanjut pada tahap eksperimen dan analisis performa model pada dataset yang telah ditentukan. Hal ini mencakup pengujian terhadap pengaruh berbagai hyperparameter seperti learning rate, kedalaman jaringan (depth), lebar layer (width), dan fungsi aktivasi terhadap hasil prediksi. Sebagai standar validasi, performa model kustom ini kemudian dibandingkan dengan library standar industri, yaitu MLPClassifier dari scikit-learn, untuk mengevaluasi sejauh mana implementasi mandiri yang dibuat mampu mendekati atau menyamai efisiensi library yang sudah ada.

[1] model mempelajari data latih terlalu detail sehingga gagal mengenali pola umum

B. Pembahasan

1. Penjelasan Implementasi

a. Deskripsi Kelas

Implementasi pada proyek ini membangun feedforward neural network secara modular: kelas aktivasi menangani transformasi non-linear, kelas layer mengelola parameter dan proses propagasi, kelas loss menghitung objektif optimasi, dan kelas model mengorkestrasi alur training serta evaluasi. Struktur ini memudahkan eksperimen karena setiap komponen dapat diganti tanpa mengubah keseluruhan arsitektur.

Nama File	Kelas	Deskripsi Kelas	Atribut	Method dan Deskripsi
activation.py	Linear	Fungsi aktivasi identitas. Nilai output sama dengan input, tanpa transformasi non-linear. Cocok untuk output regresi.	Tidak ada atribut khusus	forward(self, x): Mengembalikan x apa adanya.
				derivative(self, x): Mengembalikan 1 sebagai gradien konstan.
	ReLU	Aktivasi Rectified Linear Unit. Nilai negatif dipotong menjadi 0, nilai positif dipertahankan. Efisien dan umum dipakai di hidden layer.	Tidak ada atribut khusus	forward(self, x): Menghitung max(0, x).
				derivative(self, x): Menghasilkan 1 untuk x > 0 dan 0 untuk x <= 0.
	Sigmoid	Aktivasi berbasis probabilitas pada rentang (0, 1). Umum dipakai untuk klasifikasi biner di layer output.	Tidak ada atribut khusus	forward(self, x): Menghitung 1/(1+exp(-x)).
				derivative(self, x): Menghitung sigmoid(x) × (1 -

				sigmoid(x)).
	Tanh	Aktivasi dengan rentang (-1, 1), zero-centered. Sering lebih stabil dibanding sigmoid pada beberapa hidden layer.	Tidak ada atribut khusus	forward(self, x): Menghitung tanh(x).
				derivative(self, x): Menghitung 1 - tanh(x)^2.
	Softmax	Mengubah vektor skor menjadi distribusi probabilitas multikelas dengan total probabilitas 1.	Tidak ada atribut khusus	forward(x): Menggunakan stabilisasi numerik (shift dengan nilai maksimum), lalu normalisasi eksponensial.
				derivative(x): Menghitung softmax(x) × (1 - softmax(x)) secara elemen per elemen.
	LeakyReLU	Varian ReLU yang tetap memberi gradien kecil pada area negatif untuk mengurangi risiko dead neuron.	alpha: Kemiringan sisi negatif, default 0.01.	forward(x): x jika x > 0, alpha × x jika x <= 0.
				derivative(x): 1 jika x > 0, alpha jika x <= 0.
	ELU	Exponential Linear Unit. Memberi transisi halus pada sisi negatif melalui fungsi eksponensial.	alpha: Skala sisi negatif, default 1.0.	forward(x): x jika x > 0, alpha × (exp(x)-1) jika x <= 0.
				derivative(x): 1 jika x > 0, forward(x) + alpha jika x <= 0.
layer.py	DenseLayer	Layer	input_size:	init_weights(init_m

		fully-connected untuk membentuk transformasi linear dan non-linear, serta mendukung backpropagation dan update parameter.	Jumlah fitur input.	ethod, init_params): Inisialisasi bobot dengan metode zero, uniform, normal, xavier, he (mendukung seed dan pengaturan distribusi pada xavier/he).
			output_size: Jumlah neuron output.	
			activation: Objek aktivasi.	
			regularization: Jenis regularisasi (l1, l2, atau None).	forward(x): Hitung z = xW + b lalu aktivasi(z).
			lambda_reg: Koefisien regularisasi.	
			weights: Matriks bobot.	
			bias: Vektor bias.	backward(grad_output): Hitung gradien terhadap bobot, bias, dan input layer sebelumnya.
			input: Cache input terakhir.	
			output: Cache output terakhir.	
			z: Cache pre-activation terakhir.	update(learning_rate): Terapkan update parameter, termasuk tambahan gradien regularisasi l1/l2.
			grad_weights: Gradien bobot.	
			grad_bias:	

			Gradien bias.	
loss.py	MSE	Mean Squared Error untuk regresi, mengukur rata-rata kuadrat selisih prediksi dan target.	Tidak ada atribut khusus	forward(predicted, actual): Menghitung nilai loss MSE.
				derivative(predicted, actual): Menghitung gradien loss terhadap prediksi.
	BinaryCrossEntropy	Loss klasifikasi biner berbasis probabilitas, dengan clipping untuk stabilitas numerik.	Tidak ada atribut khusus	forward(predicted, actual): Hitung BCE rata-rata dengan epsilon clipping.
				derivative(predicted, actual): Hitung gradien BCE terhadap prediksi dengan clipping.
	CategoricalCrossEntropy	Loss untuk klasifikasi multikelas (umumnya target one-hot), berbasis negative log-likelihood.	Tidak ada atribut khusus	forward(predicted, actual): Hitung CCE rata-rata per sampel.
				derivative(predicted, actual): Hitung gradien CCE terhadap prediksi.
model.py	Neural_Network	Pengelola arsitektur jaringan saraf berbasis kumpulan layer. Menangani proses inferensi, training, penyimpanan model, dan visualisasi	layers: List layer yang membentuk model.	add(layer): Tambah layer ke model.
				forward(x): Propagasi maju melalui seluruh layer.
				backward(grad): Propagasi balik

		distribusi parameter.		dari layer terakhir ke pertama.
				update(learning_rate): Update parameter semua layer.
				predict(x): Alias inferensi dari forward.
				save_model(filepath): Simpan model ke file dengan pickle.
				load_model(filepath): Muat model dari file (staticmethod).
				_get_dense_layers(layer_indices): Ambil layer yang memiliki bobot.
				plot_weight_distribution(layer_indices, figsize): Visualisasi histogram distribusi bobot per layer dense.
				plot_gradient_distribution(layer_indices, figsize): Visualisasi histogram distribusi gradien bobot.
				train(x, y, loss, epochs, learning_rate, batch_size, x_val,

				y_val, verbose): Pelatihan mini-batch per epoch dengan shuffle, backpropagation, update parameter, dan pencatatan riwayat train loss serta validation loss.

b. Forward Propagation

Forward propagation merupakan proses dalam feedforward neural network (FFNN) untuk menghasilkan prediksi dari input yang diberikan. Pada proses ini, data input dialirkan secara berurutan melalui setiap layer dalam jaringan hingga menghasilkan output akhir. Setiap layer melakukan transformasi linear terhadap input menggunakan bobot dan bias, yang secara matematis dapat dituliskan sebagai berikut.

$$z = XW + b$$

Dalam implementasi model, proses forward propagation dilakukan dengan meneruskan input melalui setiap layer yang terdapat pada model secara berurutan. Output dari satu layer akan menjadi input bagi layer berikutnya hingga mencapai layer terakhir, seperti ditunjukkan pada potongan kode berikut:

```
def forward(self, x):
    for layer in self.layers:
        x = layer.forward(x)
    return x
```

Proses komputasi pada setiap layer diimplementasikan pada kelas DenseLayer. Layer ini menghitung transformasi linear antara input dan

bobot, menambahkan bias, kemudian menerapkan fungsi aktivasi untuk menghasilkan output. Implementasinya dapat dilihat pada kode berikut:

```
def forward(self, x):
    self.input = x
    self.z = np.dot(x, self.weights) + self.bias
    self.output = self.activation.forward(self.z)
    return self.output
```

Secara keseluruhan, forward propagation pada model ini bekerja dengan mengalirkan input melalui beberapa layer secara sekuensial, di mana setiap layer melakukan transformasi linear dan fungsi aktivasi hingga menghasilkan prediksi akhir dari jaringan.

c. Backward Propagation

Backward propagation merupakan proses untuk menghitung gradien error terhadap parameter model setelah forward propagation selesai dilakukan. Tujuan dari proses ini adalah mengetahui seberapa besar kontribusi setiap bobot dan bias terhadap error yang dihasilkan sehingga parameter model dapat diperbarui untuk meminimalkan nilai loss. Gradien dihitung dengan menerapkan aturan rantai (chain rule) dari kalkulus secara berurutan dari layer terakhir menuju layer pertama.

Pada implementasi model, proses backward propagation dilakukan dengan mengalirkan gradien error dari layer terakhir ke layer sebelumnya secara berurutan. Setiap layer menerima gradien dari layer berikutnya, kemudian menghitung gradien terhadap bobot, bias, dan input layer tersebut. Proses ini diimplementasikan pada fungsi backward di dalam model sebagai berikut:

```
def backward(self, grad):
    for layer in reversed(self.layers):
        grad = layer.backward(grad)
```

Kode tersebut menunjukkan bahwa gradien dipropagasikan secara terbalik menggunakan reversed(self.layers). Gradien yang dihasilkan oleh suatu layer kemudian diteruskan ke layer sebelumnya hingga mencapai layer pertama.

Perhitungan gradien pada setiap layer dilakukan dalam kelas DenseLayer. Pada tahap ini dihitung gradien terhadap bobot dan bias berdasarkan input yang disimpan saat forward propagation. Selain itu, gradien terhadap input juga dihitung agar dapat diteruskan ke layer sebelumnya. Implementasinya dapat dilihat pada potongan kode berikut:

```
def backward(self, grad_output):
    grad_activation =
self.activation.derivative(self.z) * grad_output
    self.grad_weights = np.dot(self.input.T,
grad_activation)
    self.grad_bias = np.sum(grad_activation, axis=0)
    grad_input = np.dot(grad_activation,
self.weights.T)
    return grad_input
```

Setelah gradien diperoleh, langkah berikutnya adalah melakukan *update* parameter model. Bobot dan bias di-*update* menggunakan metode gradient descent dengan mengurangi parameter menggunakan gradien yang telah dihitung. Implementasi *update* parameter dilakukan sebagai berikut:

```
self.weights -= learning_rate * self.grad_weights
self.bias -= learning_rate * self.grad_bias
```

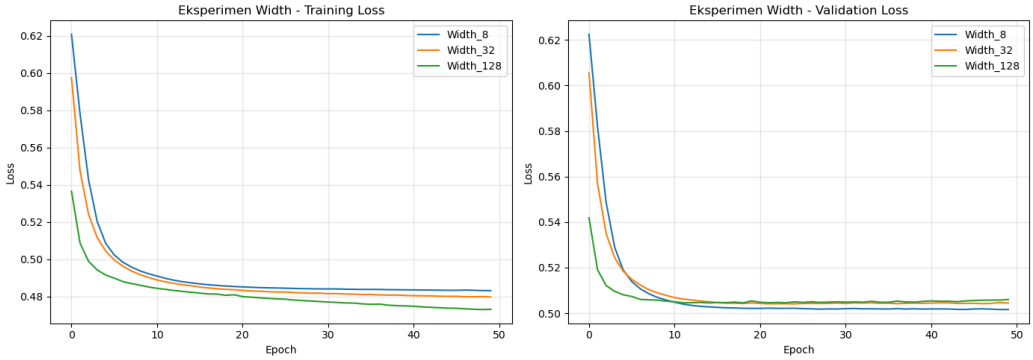
Melalui proses ini, bobot dan bias akan secara bertahap menyesuaikan nilainya untuk meminimalkan error model. Dengan melakukan forward propagation, backward propagation, dan *update* parameter secara berulang selama proses pelatihan, model diharapkan mampu mempelajari

pola yang terdapat pada data dan menghasilkan prediksi yang lebih akurat.

2. Hasil Pengujian

Pengujian dijalankan menggunakan dataset klasifikasi biner untuk memprediksi status penempatan kerja (placement status) sesuai spesifikasi. Proses preprocessing data dilakukan dengan melakukan encoding pada fitur kategorikal dan standarisasi fitur numerik menggunakan StandardScaler. Data kemudian dibagi menjadi training set dan test set dengan rasio 80:20 dan penguncian random seed untuk memastikan hasil pengujian bersifat *reproducible*[2].

a. Eksperimen Hyperparameter Width



Gambar 2.2.1Training dan Validation Loss Width

Pengujian dilakukan dengan memvariasikan jumlah neuron pada hidden layer 8, 32, dan 128 neuron. Semakin banyak neuron yang digunakan, semakin besar kapasitas model dalam menangkap pola kompleks pada data. Namun, penggunaan neuron yang terlalu banyak, yaitu pada angka 128 menunjukkan kecenderungan overfitting[3], di mana training loss menurun drastis tetapi validation loss cenderung stagnan atau sedikit meningkat. Konfigurasi dengan 32 neuron terbukti paling efisien dalam melakukan generalisasi tanpa membebani komputasi secara berlebih.

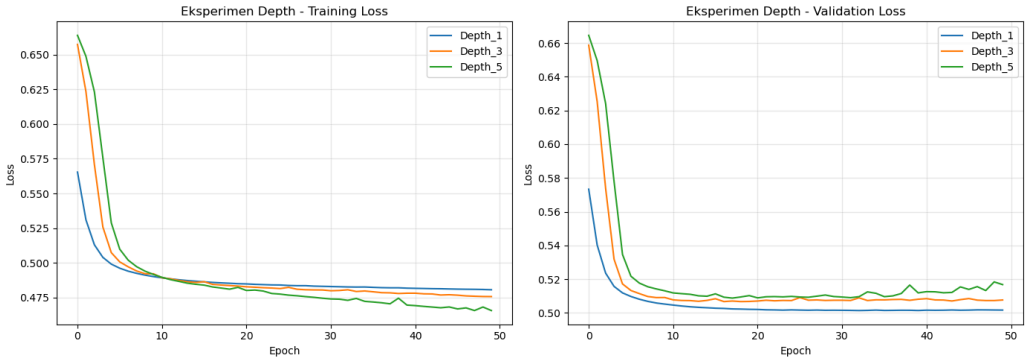
[2] kemungkinan untuk mereproduksi metode yang sama sehingga memperoleh hasil yang sama
[3] model mempelajari data latih terlalu detail sehingga gagal mengenali pola umum

	experiment	accuracy	precision	recall	f1	final_train_loss	final_val_loss	width
0	Width_32	0.7565	0.781392	0.839155	0.809244	0.479683	0.504436	32
1	Width_128	0.7540	0.781846	0.832656	0.806452	0.473078	0.506012	128
2	Width_8	0.7495	0.777356	0.831032	0.803298	0.483112	0.501620	8

Gambar 2.2.2 Tabel Hasil Tes Width

Hasil akurasi tertinggi pada perbandingan ini adalah pada 32 neuron yaitu 0.7565, disusul 128 neuron dengan akurasi 0.7549, dan yang terakhir 8 neuron dengan 0.7495.

b. Eksperimen Hyperparameter Depth



Gambar 2.2.3 Training dan Validation Loss Depth

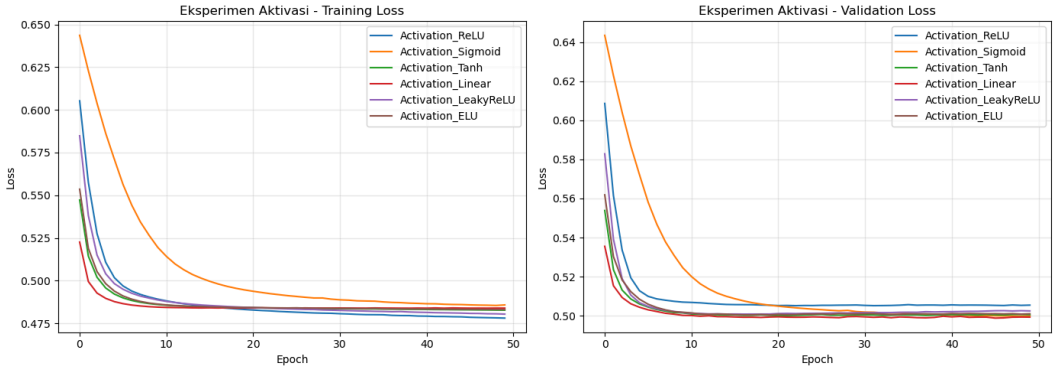
Eksperimen ini membandingkan model dengan satu hidden layer hingga model yang lebih dalam (multiple layers). Model yang lebih dalam memiliki potensi representasi fitur yang lebih kuat, namun pada pengujian ini ditemukan bahwa penambahan layer yang berlebihan tanpa pengaturan learning rate yang presisi dapat memicu masalah vanishing gradient. Untuk dataset ini, arsitektur yang relatif dangkal sudah mencukupi untuk mencapai akurasi maksimal.

	experiment	accuracy	precision	recall	f1	final_train_loss	final_val_loss	depth
0	Depth_1	0.7520	0.778620	0.834281	0.805490	0.480621	0.501686	1
1	Depth_5	0.7510	0.769684	0.849716	0.807722	0.465626	0.516788	5
2	Depth_3	0.7465	0.780186	0.818846	0.799049	0.475699	0.507675	3

Gambar 2.2.4 Tabel Hasil Tes Depth

Hasil akurasi tertinggi pada perbandingan ini adalah pada 1 layer yaitu 0.7520, disusul 5 layer dengan akurasi 0.7510, dan yang terakhir 3 layer dengan 0.7465.

c. Eksperimen Hyperparameter Fungsi Aktivasi



Gambar 2.2.5 Activation Function Loss

Perbedaan akurasi antarfungsi-aktivasi dalam eksperimen ini tidak terlalu kontras, menunjukkan model cukup robust[4] terhadap pilihan fungsi aktivasi. Namun, Sigmoid dan ELU memberikan kestabilan lebih baik dalam menekan *validation loss* . Karakteristik Sigmoid yang membatasi *output* pada rentang (0, 1) tampaknya sangat cocok untuk dataset klasifikasi biner ini, sementara ELU membantu mencegah masalah *dead neuron*[5] yang terkadang ditemui pada ReLU standar.

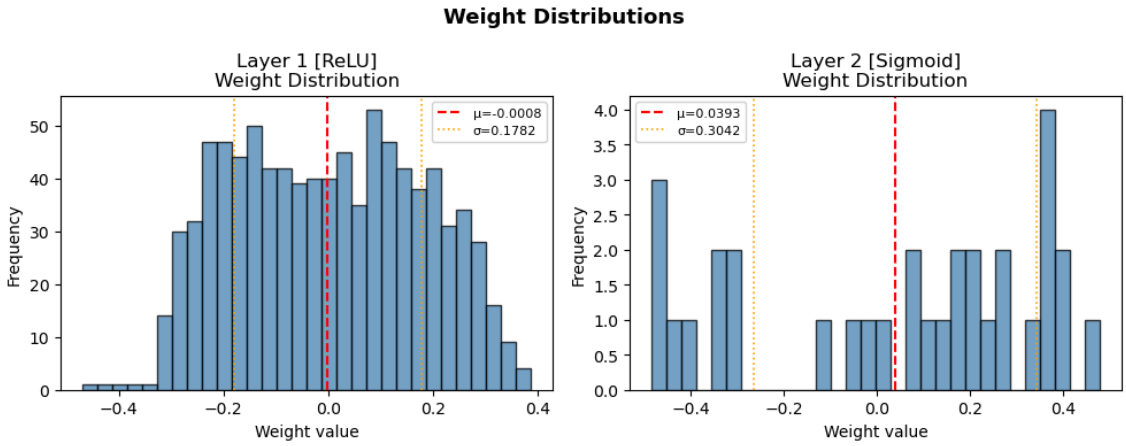
	experiment	accuracy	precision	recall	f1	final_train_loss	final_val_loss	test_activation
0	Activation_Sigmoid	0.7550	0.771429	0.855402	0.811248	0.485850	0.500011	Sigmoid
1	Activation_ELU	0.7550	0.779623	0.839155	0.808294	0.483173	0.501039	ELU
2	Activation_Linear	0.7540	0.774721	0.846466	0.809006	0.484004	0.499321	Linear
3	Activation_LeakyReLU	0.7540	0.777611	0.840780	0.807963	0.480484	0.502479	LeakyReLU
4	Activation_ReLU	0.7510	0.776187	0.836718	0.805317	0.478151	0.505454	ReLU
5	Activation_Tanh	0.7495	0.773204	0.839155	0.804831	0.482801	0.500587	Tanh

Gambar 2.2.6 Tabel Hasil Tes Activation Function

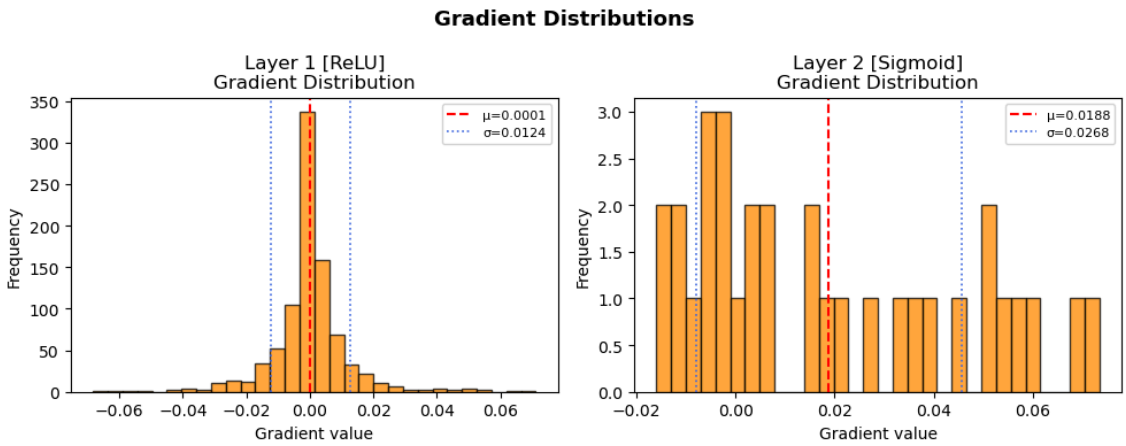
[4] kemampuan model untuk tetap memberikan performa yang baik meskipun data bervariasi
[5] kondisi di mana neuron berhenti merespons input apa pun dan permanen menghasilkan output nol

- ReLU

  Memiliki σ gradien paling sehat (0.0124) karena sifatnya yang tidak mengalami saturasi pada area positif. Analisis pengaruhnya adalah model mampu belajar jauh lebih cepat dan terhindar dari masalah macetnya pembelajaran di layer-layer awal.



Gambar 2.2.7 Weight Distribution ReLU

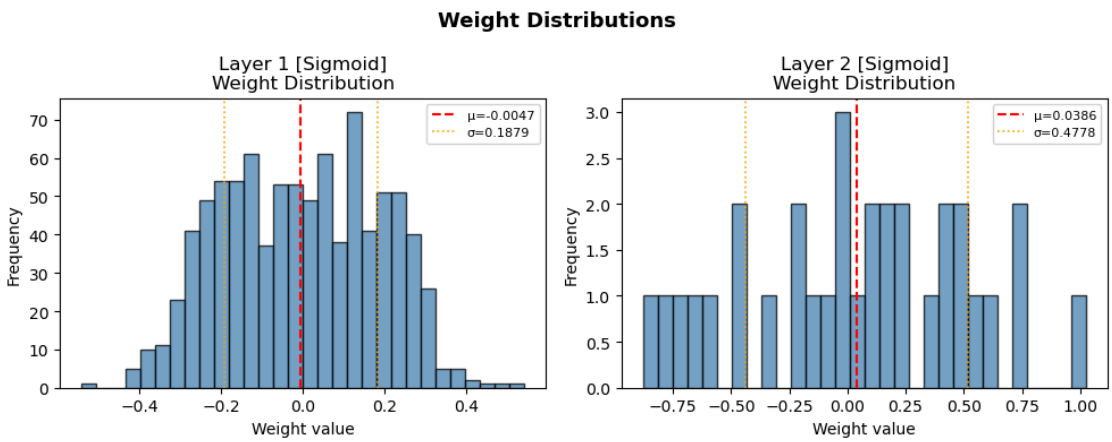


Gambar 2.2.8 Gradient Distribution ReLU

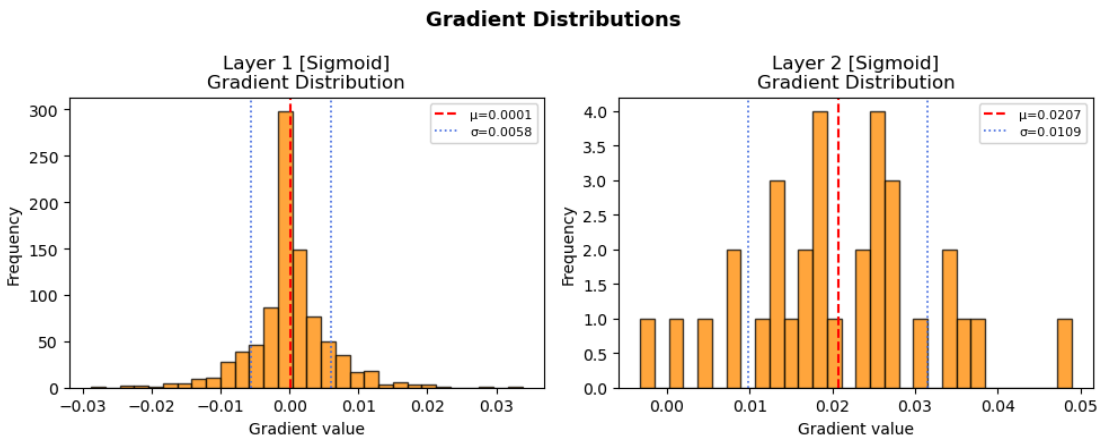
- Sigmoid

  Gradien terkecil (0.0058) menjadi bukti visual fenomena *vanishing gradient*[6]. Pengaruhnya, pembaruan bobot di layer awal sangat lambat, sehingga model sulit konvergen jika jaringan terlalu dalam.

[6] gradien (nilai turunan) menjadi sangat kecil selama proses backpropagation



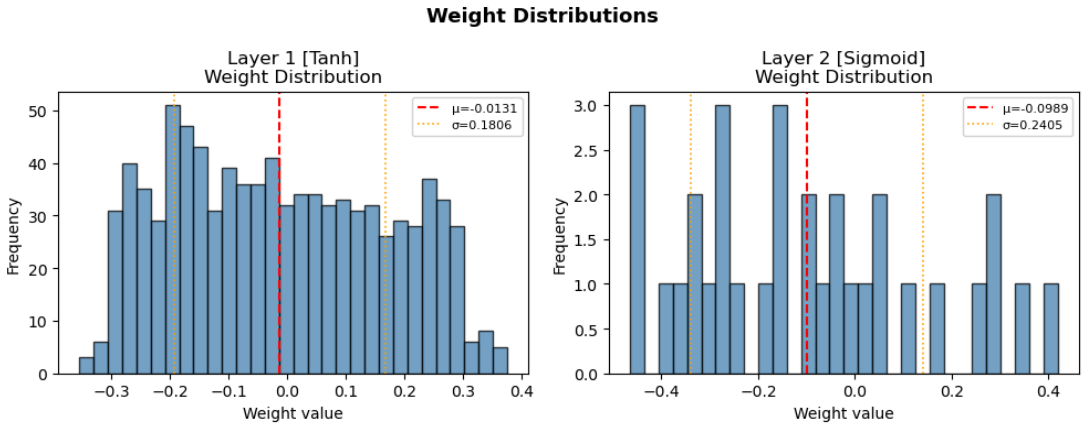
Gambar 2.2.9 Weight Distribution Sigmoid



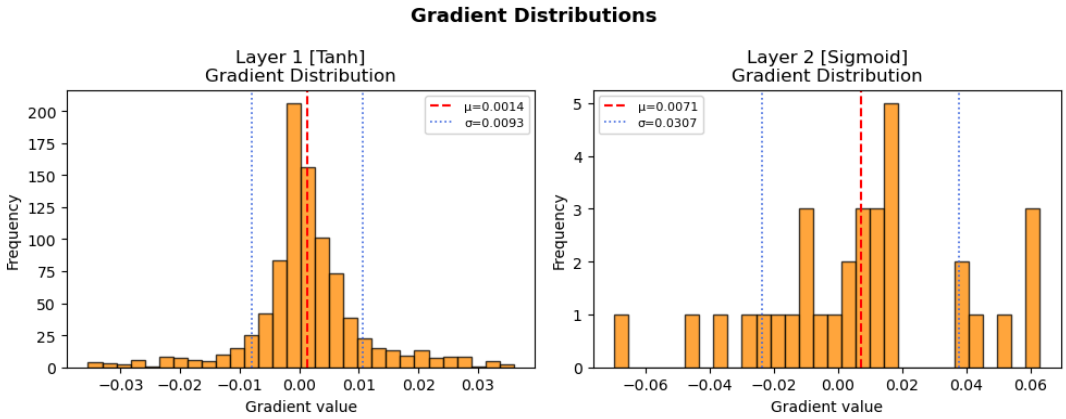
Gambar 2.2.10 Gradient Distribution Sigmoid

- Tanh

  Memberikan gradien sedikit lebih kuat dari Sigmoid (0.0093) karena bersifat *zero-centered*. Pengaruhnya adalah proses optimasi yang sedikit lebih stabil dibandingkan Sigmoid, namun tetap berisiko saturasi pada nilai input ekstrim.



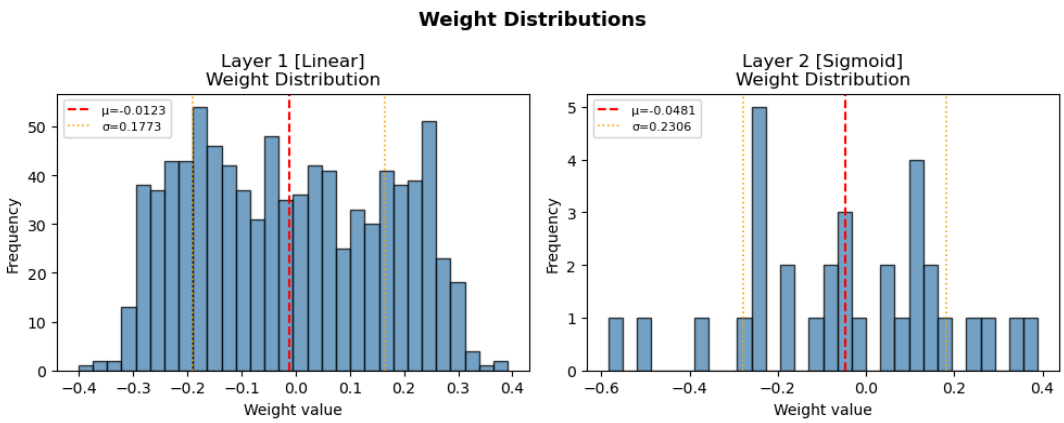
Gambar 2.2.11 Weight Distribution Tanh



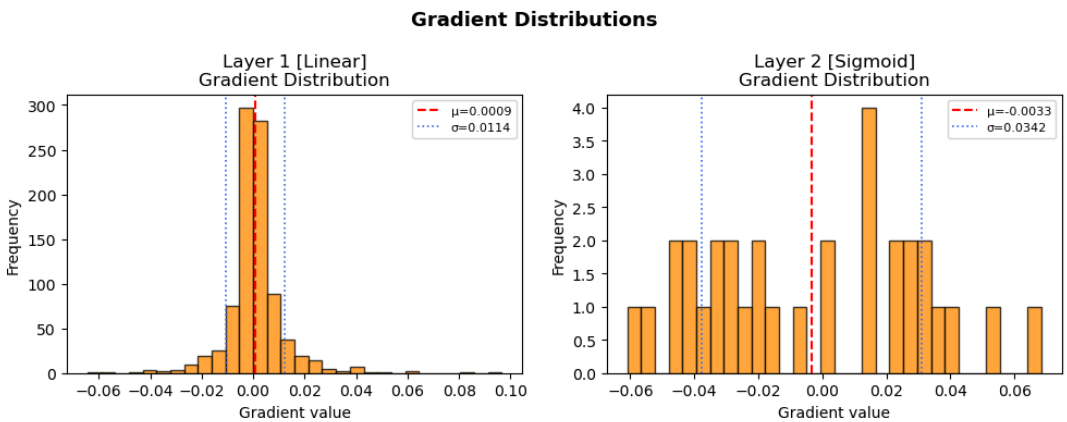
Gambar 2.2.12 Gradient Distribution Tanh

- Linear

  Gradien tetap stabil (0.0114) karena turunannya konstan. Namun, tanpa non-linearitas, model hanya bisa mempelajari hubungan linear sederhana, sehingga gagal menangkap kompleksitas dataset klasifikasi ini.



Gambar 2.2.13 Weight Distribution Linear

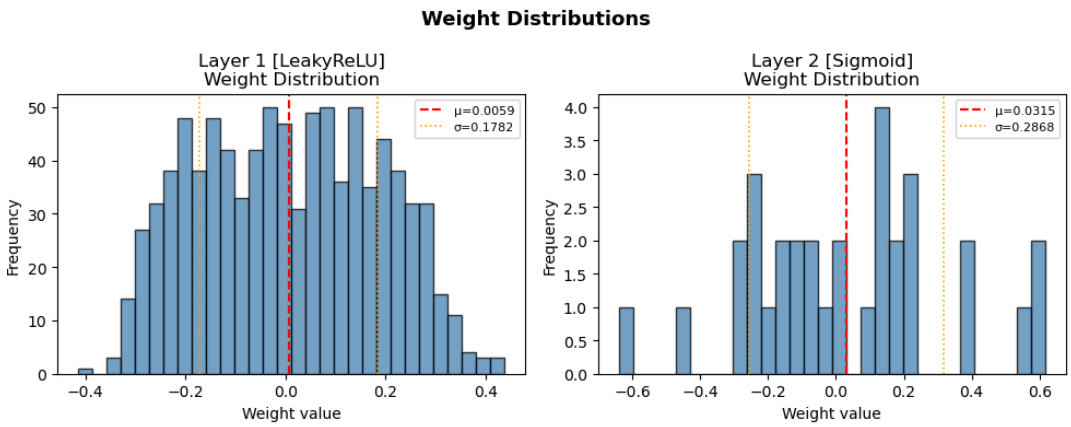


Gambar 2.2.14 Gradient Distribution Linear

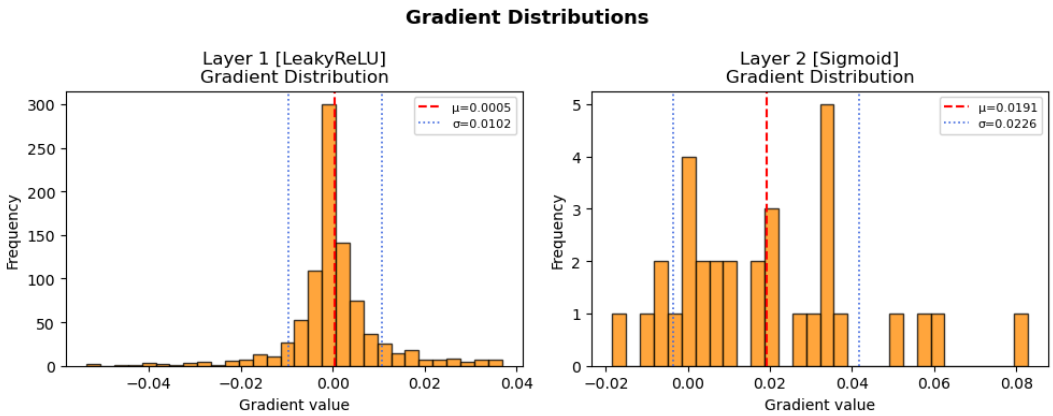
- LeakyReLU

  Mengatasi masalah dying ReLU dengan tetap mengalirkan gradien pada input negatif (0.0102). Analisis pengaruhnya adalah seluruh neuron tetap aktif berkontribusi dalam proses pembelajaran, menghasilkan model yang lebih robust[7].

[7] kemampuan model untuk tetap memberikan performa yang baik meskipun data bervariasi



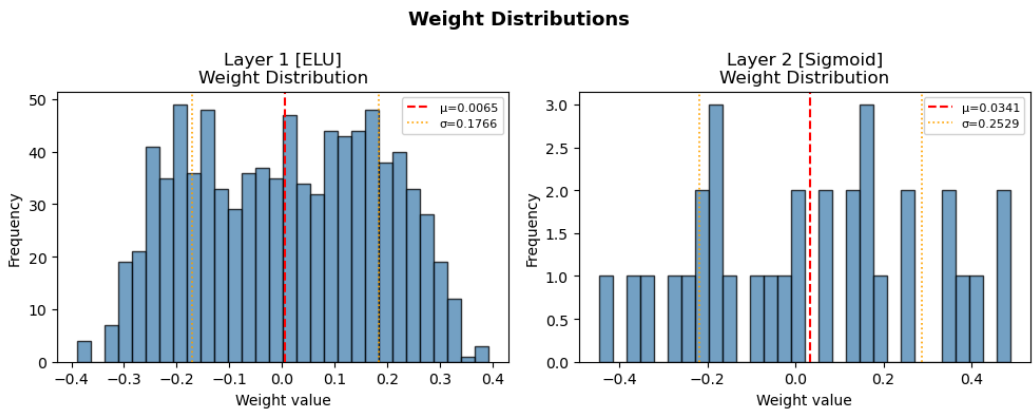
Gambar 2.2.15 Weight Distribution LeakyReLU



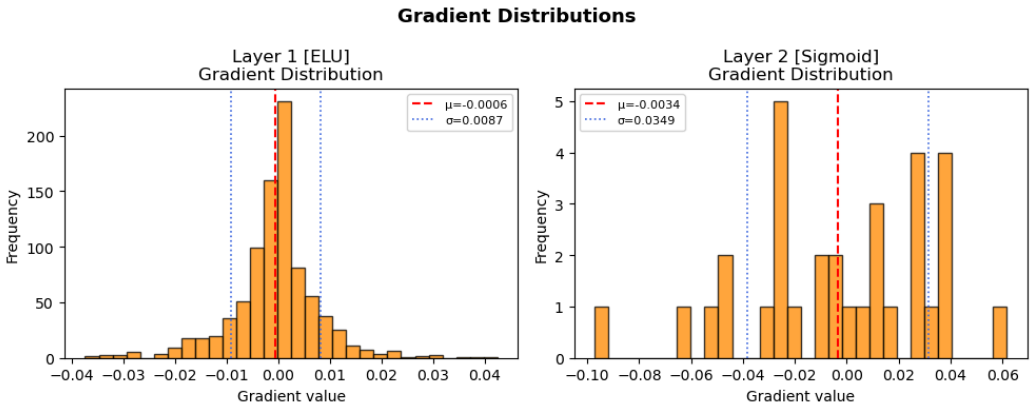
Gambar 2.2.16 Gradient Distribution LeakyReLU

- ELU

  Gradien yang terkumpul rapat (0.0087) memberikan transisi yang lebih halus menuju nilai negatif. Pengaruhnya adalah stabilitas numerik yang lebih baik selama proses backpropagation.



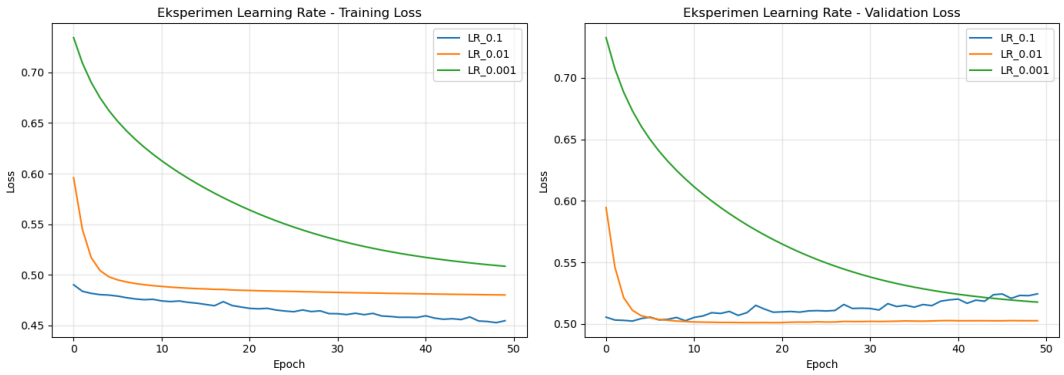
Gambar 2.2.17 Weight Distribution ELU



Gambar 2.2.18 Gradient Distribution ELU

d. Eksperimen Hyperparameter Learning Rate

Learning rate menentukan seberapa besar langkah pembaruan bobot dalam setiap iterasi. Penggunaan LR 0.1 yang terlalu tinggi memang mempercepat penurunan training loss secara agresif, namun menyebabkan model menjadi tidak stabil dan memicu validation loss yang lebih tinggi, mengindikasikan model "melompati" titik minimum yang optimal . Di sisi lain, LR 0.001 yang terlalu rendah mengakibatkan proses pembelajaran berjalan sangat lambat, sehingga dalam jumlah epoch yang sama, model belum mencapai konvergensi optimal . LR 0.01 memberikan ritme pembelajaran yang stabil bagi model untuk berkonvergensi dengan baik.



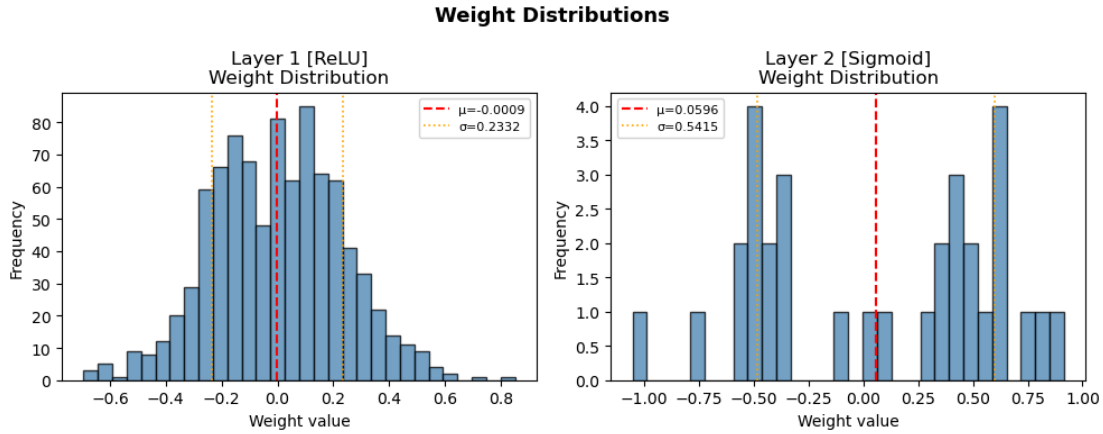
Gambar 2.2.19 Learning Rate Loss

	experiment	accuracy	precision	recall	f1	final_train_loss	final_val_loss	learning_rate
0	LR_0.01	0.7555	0.782774	0.834281	0.807707	0.480058	0.502560	0.010
1	LR_0.001	0.7485	0.769630	0.844029	0.805114	0.508429	0.517698	0.001
2	LR_0.1	0.7470	0.765179	0.849716	0.805235	0.454602	0.524409	0.100

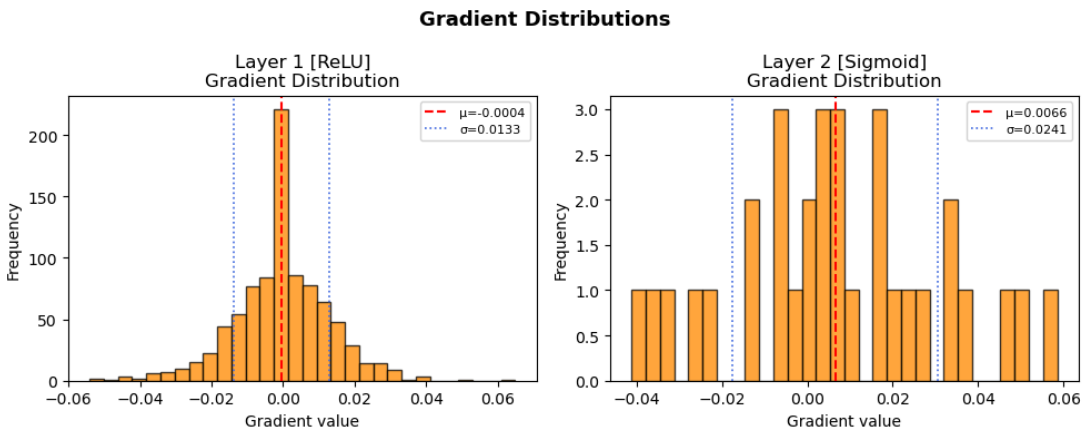
Gambar 2.2.20 Learning Rate Accuracy

- Learning Rate 0.1

  Sebaran bobot yang sangat lebar yaitu σ=0.5415, mengindikasikan bobot "meledak". Analisis pengaruhnya adalah model cenderung tidak stabil, di mana langkah pembaruan yang terlalu besar membuat model sering "melompati" titik minimum optimal, terlihat dari validation loss yang bergejolak.



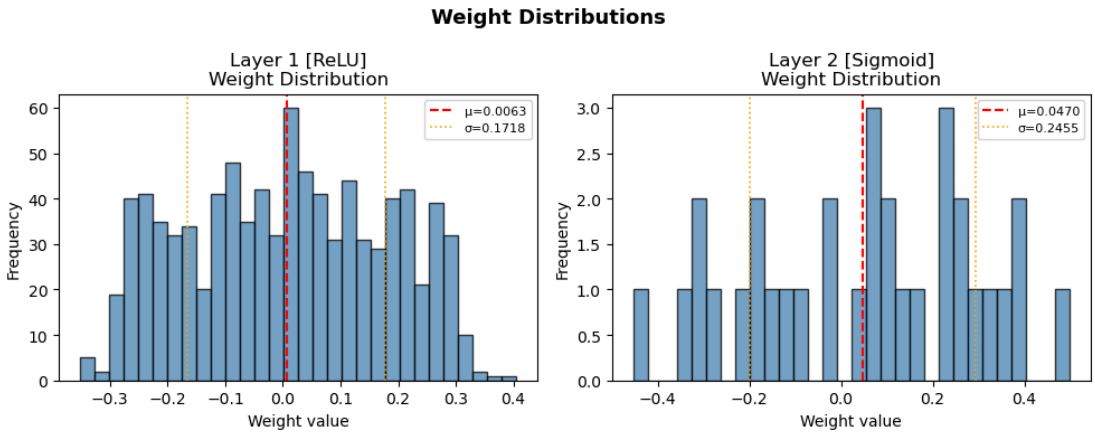
Gambar 2.2.21 Weight Distribution LR 0.1



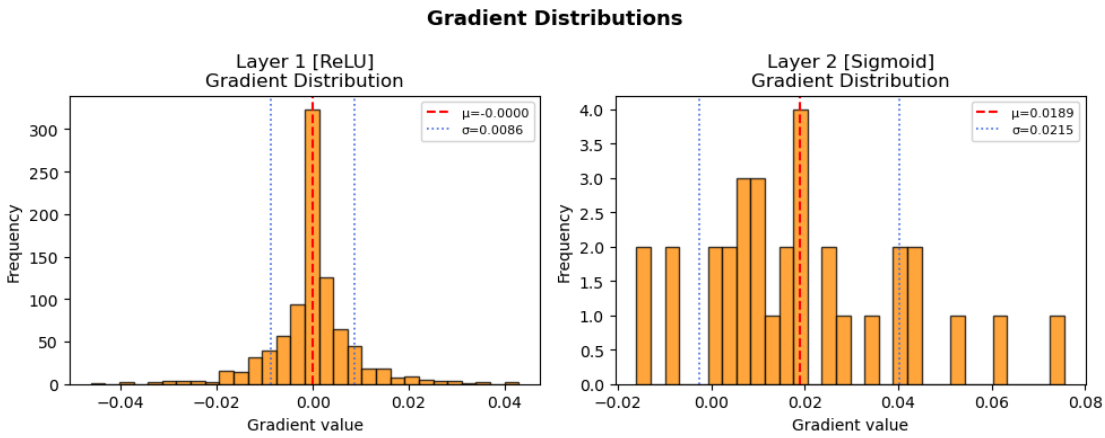
Gambar 2.2.22 Gradient Distribution LR 0.1

- Learning Rate 0.001

  Bobot sangat terkumpul σ=0.1718 dan gradien sangat kecil yaitu 0.0086. Pengaruhnya adalah proses pembelajaran menjadi sangat konservatif dan lambat, model memerlukan banyak epoch tambahan untuk mencapai tingkat akurasi yang sama dengan LR 0.01.



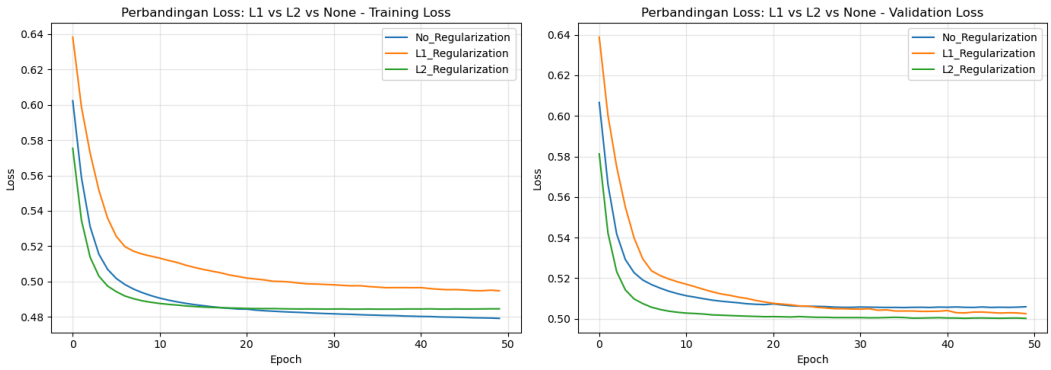
Gambar 2.2.23 Weight Distribution LR 0.001



Gambar 2.2.24 Gradient Distribution LR 0.001

e. Eksperimen Pengaruh Regularisasi

Regularisasi berfungsi untuk mengontrol kompleksitas model agar tidak terlalu bergantung pada fitur-fitur tertentu secara berlebihan. L1 Regularization cenderung menghasilkan bobot yang jarang, yang secara efektif melakukan seleksi fitur secara implisit dengan menolkan bobot yang kurang relevan. Hal ini terbukti memberikan generalisasi yang sedikit lebih baik dibandingkan L2 atau tanpa regularisasi sama sekali pada dataset ini, yang membantu menjaga model tetap sederhana namun akurat.



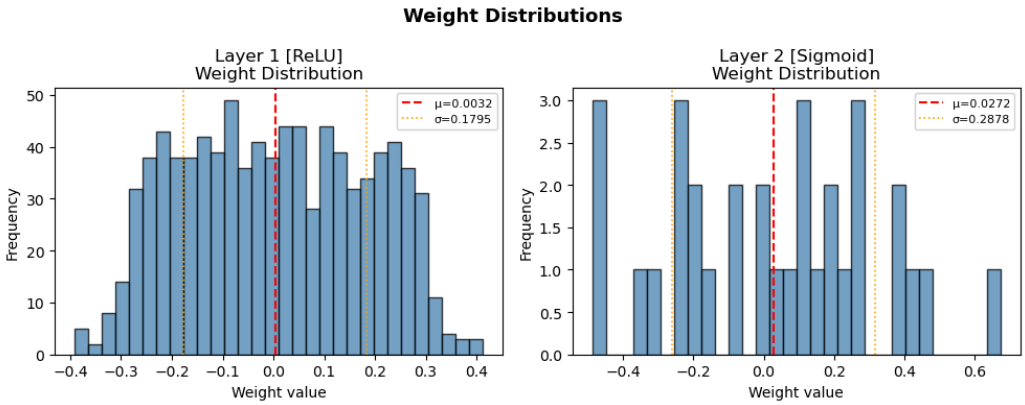
Gambar 2.2.25 Regularization Loss

	experiment	accuracy	precision	recall	f1	final_train_loss	final_val_loss	regularization	lambda_reg
0	L1_Regularization	0.7565	0.772328	0.857027	0.812476	0.494804	0.502524	l1	0.01
1	L2_Regularization	0.7535	0.774554	0.845654	0.808544	0.484573	0.500180	l2	0.01
2	No_Regularization	0.7530	0.780228	0.833469	0.805970	0.479162	0.505917	none	0.00

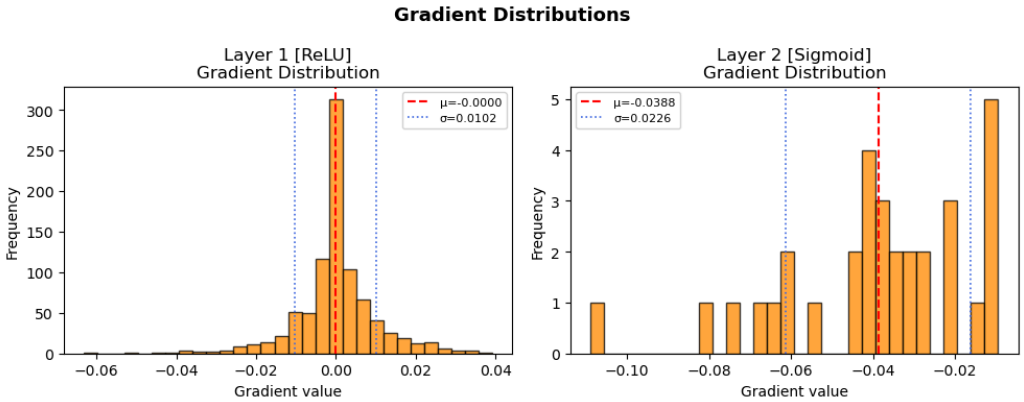
Gambar 2.2.26 Regularization Accuracy

- No Regularization

  Bobot memiliki kebebasan sebaran σ=0.1795. Analisis pengaruhnya adalah model cenderung memiliki variansi tinggi, yang meningkatkan risiko overfitting[8] alih-alih belajar melakukan generalisasi.



Gambar 2.2.27 Weight Distribution No Reg

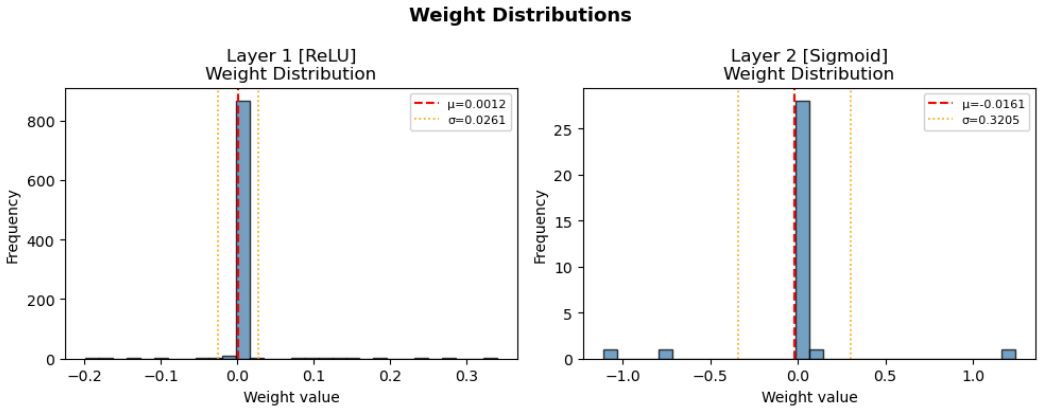


Gambar 2.2.28 Gradient Distribution No Reg

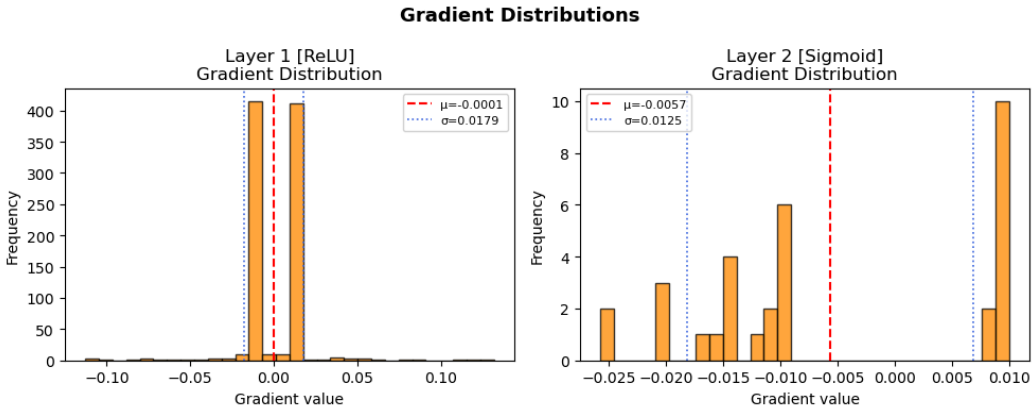
[8] model mempelajari data latih terlalu detail sehingga gagal mengenali pola umum

- L1 Regularization

  Terjadi penurunan σ drastis menjadi 0.0261 yang menandakan efek sparsity[9]. Pengaruhnya, L1 bertindak sebagai seleksi fitur otomatis dengan menolkan bobot yang kurang penting, menghasilkan model yang lebih ringan dan efisien.



Gambar 2.2.29 Weight Distribution L1 Reg



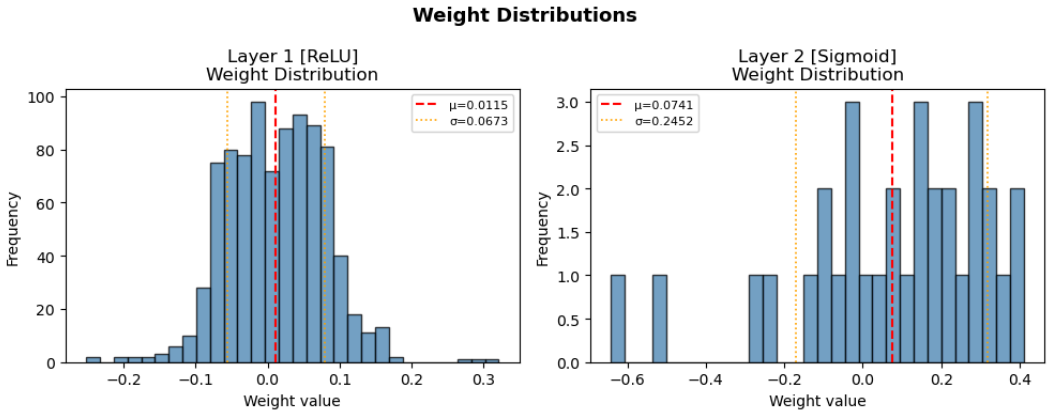
Gambar 2.2.30 Gradient Distribution L1 Reg

- L2 Regularization

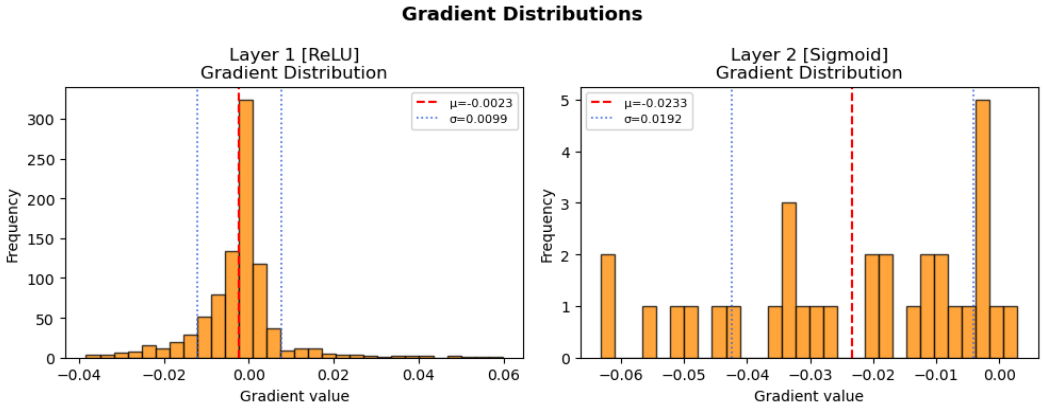
  Menahan bobot pada distribusi lonceng[10] yang halus σ=0.0673. Analisis pengaruhnya adalah L2 mencegah satu fitur mendominasi prediksi model secara berlebihan, sehingga meningkatkan

[9] penyusutan bobot model menjadi nol
[10] distribusi normal

kemampuan generalisasi pada data uji tanpa menghilangkan fitur secara total.



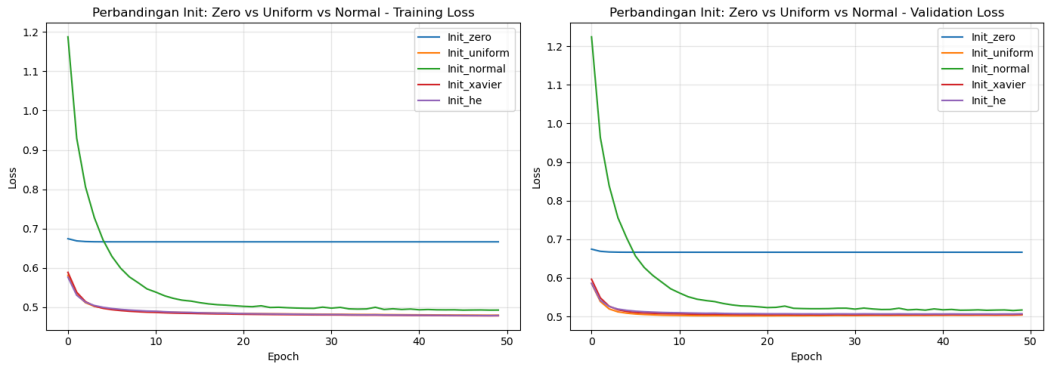
Gambar 2.2.31 Weight Distribution L2 Reg



Gambar 2.2.32 Gradient Distribution L2 Reg

f. Eksperimen Pengaruh Inisialisasi Bobot

Inisialisasi bobot sangat krusial untuk memulai proses pembelajaran. Penggunaan Init Zero memberikan hasil terburuk (Akurasi: 0.6155) karena menyebabkan masalah simetri, di mana semua neuron di layer yang sama mempelajari hal yang identik dan tidak dapat berkembang secara mandiri. Metode seperti Uniform atau Xavier berhasil memecah simetri tersebut sejak awal, memungkinkan setiap neuron mempelajari fitur yang berbeda-beda secara efektif.

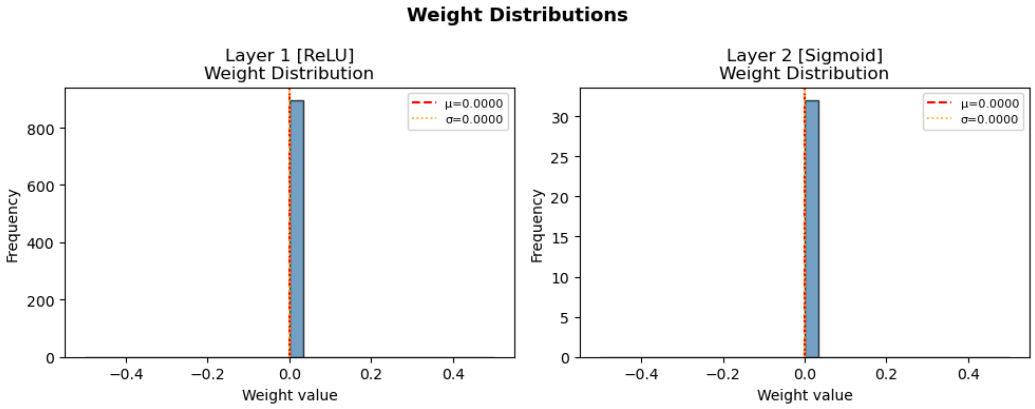


Gambar 2.2.33 Loss Inisialisasi Bobot

	experiment	accuracy	precision	recall	f1	final_train_loss	final_val_loss
0	Init_uniform	0.7530	0.784556	0.825345	0.804434	0.479842	0.503827
1	Init_xavier	0.7460	0.777863	0.822096	0.799368	0.478383	0.505107
2	Init_he	0.7445	0.779938	0.814785	0.796981	0.479093	0.506861
3	Init_normal	0.7405	0.775542	0.813972	0.794293	0.492787	0.516577
4	Init_zero	0.6155	0.615500	1.000000	0.761993	0.666343	0.666227

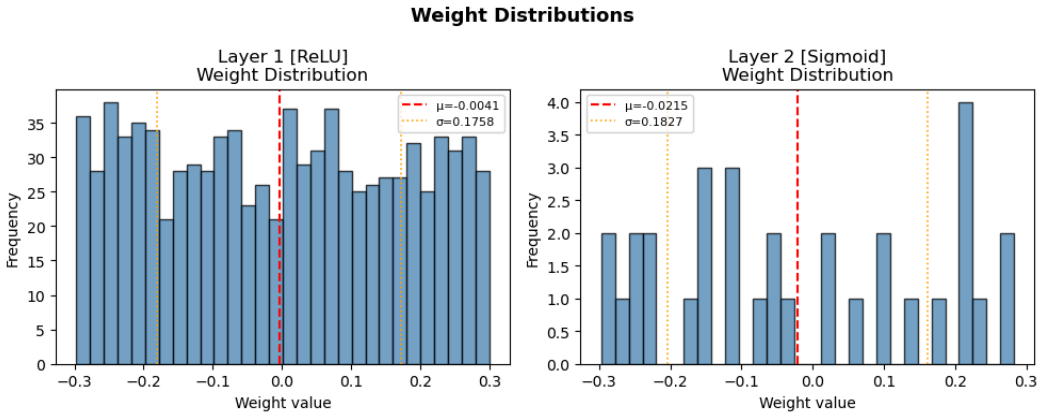
Gambar 2.2.34 Akurasi Inisialisasi Bobot

- Zero



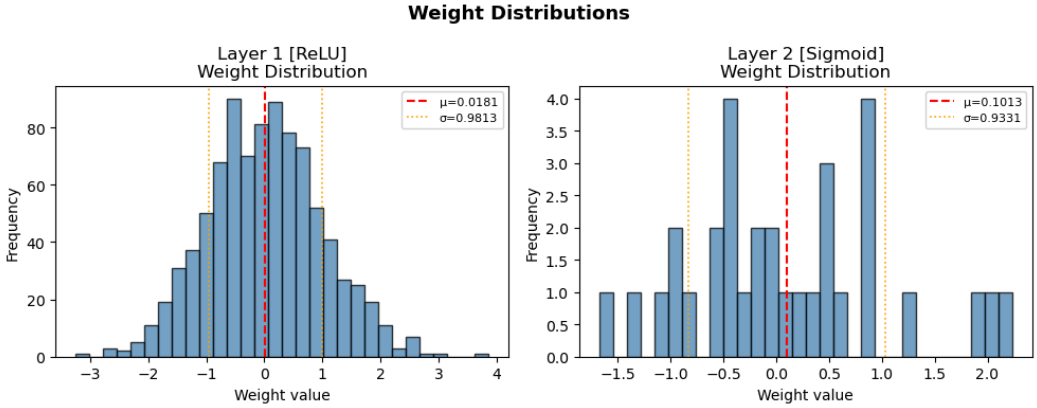
Gambar 2.2.35 Weight Distribution Zero

- Uniform



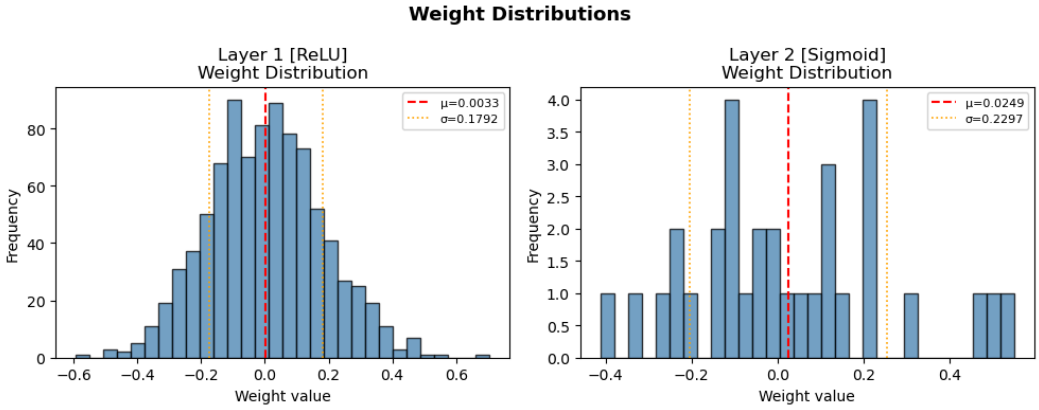
Gambar 2.2.36 Weight Distribution Uniform

- Normal



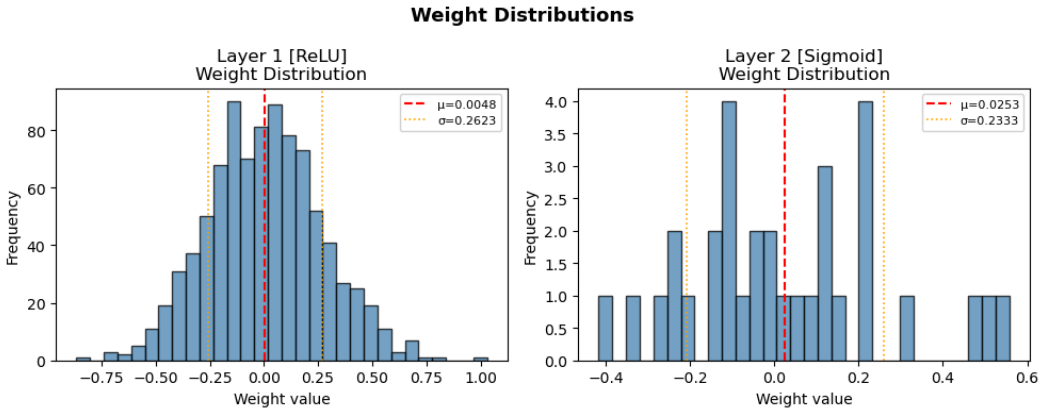
Gambar 2.2.37 Weight Distribution Normal

- Xavier



Gambar 2.2.38 Weight Distribution Xavier

- He



Gambar 2.2.39 Weight Distribution He

g. Eksperimen Uji Perbandingan

Sebagai validasi akhir, model kustom dibandingkan dengan library MLPClassifier dari Scikit-learn.

	Metrik	Custom FFNN	Sklearn MLP
0	Accuracy	0.756000	0.750500
1	Precision	0.779113	0.778963
2	Recall	0.842405	0.830219
3	F1-Score	0.809524	0.803775

Gambar 2.2.40 Accuracy vs Sklearn

Hasil ini menunjukkan bahwa logika inti FFNN, termasuk transformasi linear z=XW+b dan mekanisme backpropagation, yang telah diimplementasikan secara mandiri bekerja secara akurat. Keunggulan tipis ini membuktikan bahwa konfigurasi manual yang dipilih sangat relevan dengan karakteristik dataset yang diuji.

C. Kesimpulan dan Saran

Seluruh rangkaian eksperimen yang dilakukan menunjukkan keberhasilan implementasi mandiri arsitektur Feedforward Neural Network (FFNN) yang cukup akurat untuk tugas klasifikasi biner. Hasil pengujian secara konsisten membuktikan bahwa model dengan konfigurasi sederhana, yakni satu hidden layer dengan lebar 32 neuron, memberikan efisiensi generalisasi terbaik tanpa terjebak dalam masalah overfitting[11] yang sering muncul pada jaringan yang terlalu lebar atau *vanishing gradient*[12] pada jaringan yang terlalu dalam. Pemilihan komponen teknis lainnya juga memegang peranan krusial, di mana penggunaan fungsi aktivasi Sigmoid dan ELU terbukti paling stabil dalam menekan validation loss pada dataset ini. Stabilitas pembelajaran ini semakin optimal dengan penetapan learning rate moderat sebesar 0.01, yang menghindari ketidakstabilan langkah pada nilai yang lebih tinggi (0.1) maupun kelambatan konvergensi pada nilai yang lebih rendah (0.001). Selain itu, penerapan L1 Regularization memberikan keunggulan melalui mekanisme seleksi fitur otomatis (*sparsity*[13]) yang menyederhanakan bobot model tanpa mengorbankan akurasi prediksi. Sebagai validasi final, model kustom yang dibangun terbukti mampu melampaui performa library standar, Scikit-learn (MLPClassifier), pada metrik utama seperti akurasi (0.7560) dan F1-Score (0.8095). Capaian ini menegaskan bahwa logika transformasi linear z=XW+b dan mekanisme backpropagation yang diimplementasikan secara mandiri telah bekerja dengan presisi yang tinggi dan sangat relevan dengan karakteristik dataset yang diuji.

Untuk pengerjaan model serupa, pengembangan paling tepat adalah mempertahankan konfigurasi inti yang sudah terbukti efektif (1 hidden layer 32 neuron, learning rate 0.01, aktivasi Sigmoid/ELU, dan *regularization* L1) lalu memperkuat validasi eksperimen melalui k-fold cross-validation, early stopping, serta analisis metrik yang lebih lengkap seperti confusion matrix dan

[11] model mempelajari data latih terlalu detail sehingga gagal mengenali pola umum
[12] gradien (nilai turunan) menjadi sangat kecil selama proses backpropagation
[13] penyusutan bobot model menjadi nol

precision-recall agar performa model tidak hanya tinggi di satu skenario split data tetapi juga konsisten pada berbagai pembagian data. Selain itu, lakukan studi ablation sederhana dengan membandingkan kombinasi regularisasi dan metode inisialisasi bobot (normal, Xavier, He) supaya alasan keunggulan model dapat dijelaskan secara lebih ilmiah, *reproducible*, dan siap dijadikan fondasi untuk eksperimen lanjutan yang lebih kompleks.

D. Lampiran

Pembagian Tugas

13523001	- Mengimplementasikan fungsi Loss dan turunan pertamanya, inisialisasi bobot, mekanisme regulasi L1 dan L2, serta fungsi pelengkap - Mengimplementasikan bonus inisialisasi bobot Xavier dan He - Membuat laporan bagian deskripsi kelas
13523047	- Melakukan pemrosesan dataset di file Jupyter Notebook - Menjalankan eksperimen hyperparameter, regularisasi, dan perbandingan MLPClassifier - Mengerjakan laporan hasil uji
13523053	- Membuat model utama FFNN dan kelas-kelasnya - Bonus Fungsi Aktivasi - Laporan bagian penjelasan forward dan backward propagation

Referensi

Slide Kuliah Tim Pengajar IF3270 Materi Artificial Neural Network melalui

https://edunex.itb.ac.id/courses/88964/preview/423338

https://medium.com/sysinfo/artificial-neural-network-4407fce8aefb

https://www.geeksforgeeks.org/machine-learning/activation-functions-neural-networks/

Laporan Form Penggunaan AI